

Cómo funciona este proyecto, paso a paso

Una guía accesible sobre la resolución del Desafío Práctico (Push/Fold con un Algoritmo Genético)

*Una explicación pensada para **cualquier persona**, sin necesidad de saber de algoritmos evolutivos ni de programación. La idea es entender qué hacemos, por qué y cómo llegamos al resultado, sin meternos en las cuentas finas.*

La idea en 30 segundos

Queremos encontrar la **mejor forma de jugar** una situación concreta de póker. En vez de calcularla a mano, dejamos que una idea prestada de la **naturaleza** la descubra sola: creamos muchas estrategias al azar, dejamos que «sobrevivan» las mejores, que se «reproduzcan» y «muten», y repetimos ese ciclo muchas veces. Generación tras generación, las estrategias mejoran hasta dar con la óptima. A eso se lo llama **Algoritmo Genético**.

Piensen en la evolución de las especies, pero aplicada a estrategias de póker y acelerada a miles de «años» por segundo dentro de la computadora.

Parte 1 — El problema, en lenguaje humano

El póker de la situación

Imaginen una partida de póker **uno contra uno** (en la jerga, *heads-up*) donde a los dos jugadores les quedan **pocas fichas**. Cuando el stack (las fichas que tenés) es chico, jugar con matices —apostar poco, ver cartas, farolear— casi no tiene sentido: el riesgo es alto y el margen para maniobrar, mínimo. Por eso la jugada se reduce a una **decisión de sí o no**:

- **Push** → apostar *todas* las fichas de una (ir «all-in»).
- **Fold** → tirar la mano y no jugar.

El jugador recibe sus dos cartas y, sin más vueltas, decide: **¿push o fold?**

¿Por qué es un problema interesante?

Porque no da igual con qué cartas hacés cada cosa. Con un par de ases, obviamente vas all-in. Con un 7 y un 2 de distinto palo (la peor mano del póker), obviamente tirás. ¿Pero qué hacés con las manos «del medio», como una jota con un diez? Ahí está la gracia.

En el póker hay **169 tipos de mano inicial distintos**. Una **estrategia completa** es una decisión —push o fold— para *cada una* de esas 169 manos. ¿Cuántas estrategias posibles hay? Como cada mano tiene 2 opciones y son 169 manos, el total es **2^{169}** , un número con más de **50 cifras**: muchísimo más grande que la cantidad de átomos que hay en la Tierra. **Probarlas todas una por una es imposible**, ni con todas las computadoras del mundo.

Y acá aparece la pregunta central del proyecto: entre esas 2^{169} estrategias posibles, ¿cuál es la que más fichas gana a la larga? Buscar la mejor aguja en un pajar tan gigante es justo el tipo de problema donde brillan los **algoritmos evolutivos**.

Parte 2 — ¿Qué es un Algoritmo Genético?

Es una técnica de búsqueda inspirada en la **selección natural** de Darwin. En lugar de razonar la respuesta, la hacemos *evolucionar*. Los ingredientes son los mismos que en la biología:

En la naturaleza	En nuestro algoritmo
Un individuo (un bicho)	Una estrategia de póker
Su ADN (cromosoma)	La lista de 169 decisiones push/fold
Una población	Un montón de estrategias conviviendo
Qué tan apto es para sobrevivir	Cuántas fichas gana la estrategia
Reproducción (los hijos heredan)	Combinar dos estrategias buenas en una nueva
Mutación (cambios al azar)	Cambiar alguna decisión suelta al azar
Una generación	Una vuelta completa del ciclo

La regla de oro es la misma que en la evolución: **los más aptos tienen más chances de dejar descendencia**. Si repetimos el ciclo muchas veces, la población entera va mejorando, porque las buenas características se acumulan y las malas se descartan.

Lo importante: el algoritmo **no sabe nada de póker**. Solo sabe comparar estrategias por cuántas fichas ganan y aplicar estas reglas de «supervivencia del más apto». Aun así, termina descubriendo una forma de jugar muy sensata. Eso es lo fascinante.

Parte 3 — Cómo lo resolvimos, paso a paso

Paso 0 — Preparar un «árbitro» que sepa quién gana

Antes de que el algoritmo empiece a evolucionar, necesitamos algo que le diga, para cualquier enfrentamiento de cartas, **qué tan probable es ganar**. Por ejemplo: un par de ases le gana a un par de reyes el **82%** de las veces. Calculamos por adelantado una **gran tabla** con la probabilidad de ganar de cada mano contra cada otra (169×169). La calculamos una sola vez —simulando muchísimos repartos— y la guardamos; a partir de ahí el algoritmo la consulta al instante, como quien mira un vademécum. En el proyecto la llamamos **matriz de equity**.

Paso 1 — Escribir una estrategia como una lista de ceros y unos

Cada estrategia se representa como una **grilla de 169 casilleros**, uno por cada tipo de mano. En cada casillero ponemos **1** si con esa mano vamos a hacer **push**, o **0** si vamos a hacer **fold**. Esta grilla es exactamente el «ADN» de la estrategia: los jugadores ya usan estas tablas 13×13 para estudiar; acá las volvemos números que la computadora puede manipular.

Paso 2 — Medir qué tan buena es una estrategia (el «fitness»)

Para comparar estrategias necesitamos un puntaje. El nuestro es muy natural: **¿cuántas fichas gana (o pierde), en promedio, esta estrategia por cada mano jugada?** Lo calculamos recorriendo las 169 manos y, según lo que diga la grilla, sumando las fichas esperadas en cada caso (usando el árbitro del Paso 0 y con qué manos nos paga el rival). El resultado es un único número, el **fitness**: cuanto más alto, mejor. Ese es el puntaje que el algoritmo intenta maximizar.

Paso 3 — Arrancar con una población al azar

Creemos **80 estrategias completamente aleatorias**. Casi todas van a ser malísimas —hacen all-in con basura y tiran manos buenas—, y eso está perfecto: son el «caldo primitivo» del que va a partir la evolución.

Paso 4 — El ciclo de una generación

Con la población actual hacemos cuatro cosas:

- **1. Selección (los más aptos «sobreviven»)**. Elegimos parejas para que se reproduzcan, favoreciendo a las de mayor fitness mediante un mini-torneo: tomamos unas pocas al azar y gana la mejor.
- **2. Cruza (reproducción)**. De cada pareja nace una estrategia «hija» que hereda algunas decisiones de un padre y otras del otro; combinar dos buenas puede dar una mejor.
- **3. Mutación (variación al azar)**. A la hija le cambiamos alguna decisión suelta al azar. Esto introduce novedad y evita que la población se estanque.
- **4. Elitismo (no perder lo mejor)**. Las **2 mejores** estrategias pasan intactas a la siguiente generación, así nunca damos un paso atrás.

El resultado es una **nueva generación** de 80 estrategias, en promedio un poquito mejores que las anteriores.

Paso 5 — Repetir muchas veces

Repetimos el Paso 4 **120 veces** (120 generaciones). Al principio las mejoras son enormes; después, cada vez más finas, hasta que la población deja de mejorar porque ya encontró la mejor forma de jugar. En ese punto, paramos.

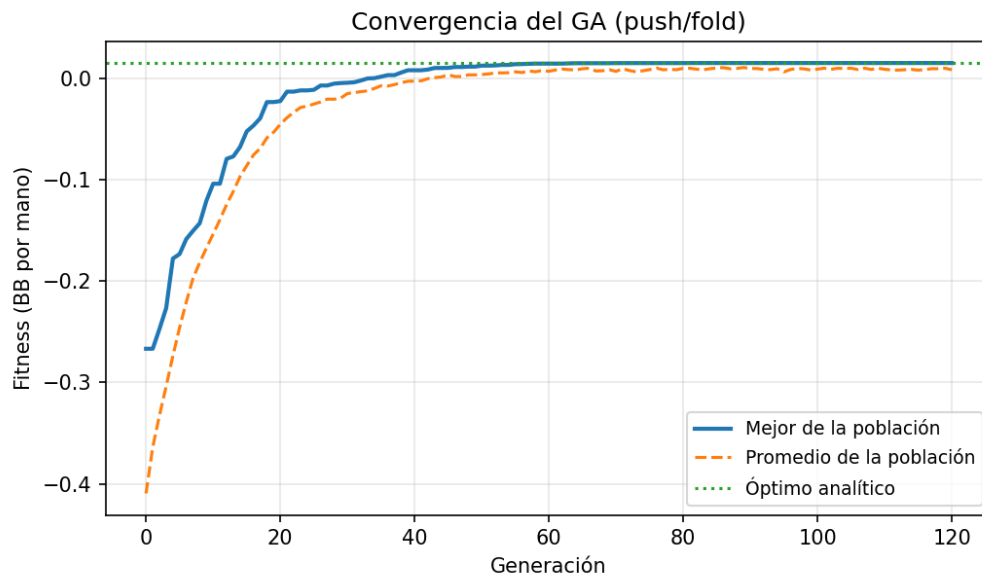
Paso 6 — Verificar que la respuesta sea la correcta

¿Cómo sabemos que es de verdad lo óptimo? En esta versión el rival juega de una forma fija, así que la respuesta perfecta se puede **calcular directamente con una cuenta** (mano por mano, conviene ir all-in solo si eso gana más fichas que tirar). Comparamos la estrategia evolucionada contra esa respuesta calculada y coinciden en **las 169 manos**: el Algoritmo Genético llegó, solo y sin saber póker, exactamente a la jugada óptima. Esa es nuestra prueba de que todo funciona.

Parte 4 — Cómo leer los resultados

1. La curva de convergencia — «cómo aprende el algoritmo»

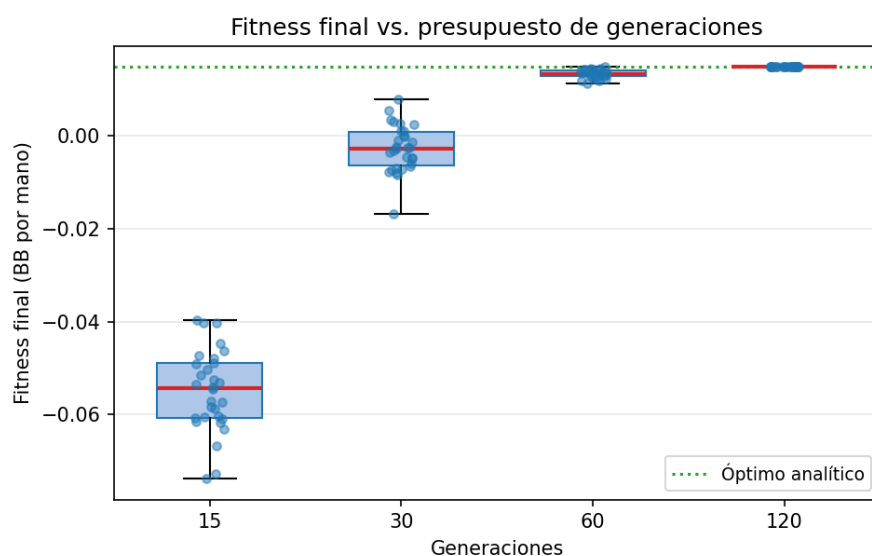
El eje horizontal son las generaciones y el vertical, el fitness (fichas por mano; **más arriba es mejor**). Las estrategias arrancan muy abajo (perdiendo, porque son al azar) y **suben rápido** hasta pegarse a la línea verde punteada, que marca la jugada óptima. Esa «subida que se aplana» es la firma de un algoritmo que **converge**.



La curva de convergencia: el algoritmo mejora mucho al principio y luego se estabiliza en la mejor respuesta.

2. El diagrama de caja — «qué tan confiable es»

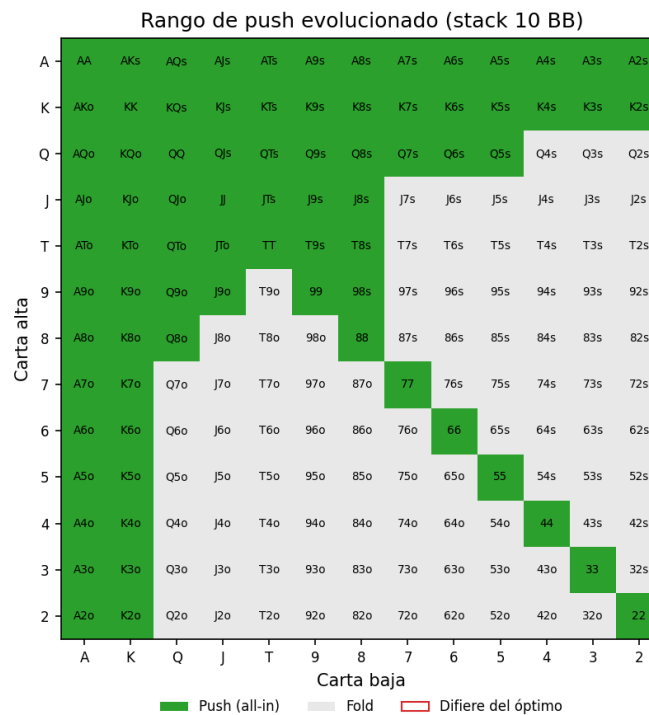
Corrimos el experimento **30 veces** (cada una desde un azar distinto) y miramos qué tan bien le va según cuántas generaciones la dejemos correr. Con **más generaciones**, las cajas suben hacia la línea óptima y se vuelven **más chiquitas**: no solo mejora el promedio, sino que **todas** las corridas terminan dando lo mismo. El método es **consistente**, no depende de la suerte.



Con más generaciones, todas las corridas convergen al óptimo y la dispersión desaparece.

3. La grilla de la estrategia — «el resultado, en idioma póker»

La estrategia ganadora traducida de vuelta al póker. En verde, las manos con las que conviene ir **all-in**; en gris, las que conviene **tirar**. El resultado tiene todo el sentido del mundo: all-in con **todos los pares** y con **cualquier mano que tenga un as o un rey** (sí, incluso K2), más las manos altas y algunos conectores del mismo palo; y se tiran las manos bajas y descoordinadas. Que hasta K2 entre no es porque sea fuerte, sino porque —con el rival foldeando casi la mitad de las veces y la ciega ya posteadada— **rendirse pierde todavía más**. En total, push con alrededor del **43%** de las manos. El algoritmo redescubrió, por su cuenta, una tabla que los jugadores tardaron años en pulir.



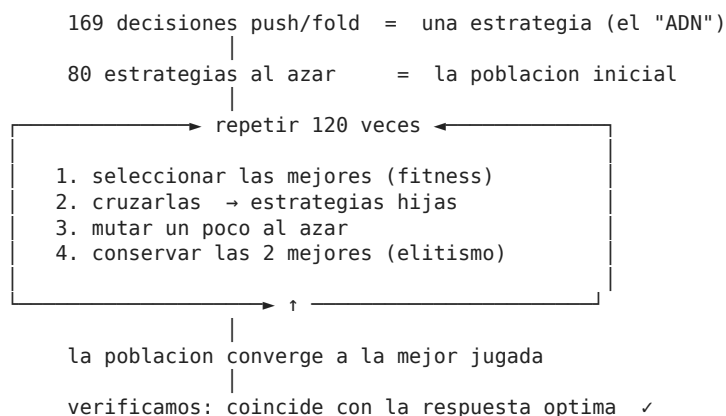
La estrategia óptima hallada por el algoritmo, en la clásica grilla 13×13 del póker (verde = push, gris = fold).

Parte 5 — Qué aprendimos (y qué falta)

Lo importante: un procedimiento que solo sabe «quedarse con lo mejor y variar un poco», repetido muchas veces, es capaz de resolver un problema con más combinaciones que átomos en el planeta y llegar a una respuesta **correcta y con sentido**. Esa es la potencia de los algoritmos evolutivos.

La limitación honesta: en esta versión el rival juega de una forma **fija**. Eso hace el problema más sencillo (por eso pudimos verificar la respuesta con una cuenta directa). El paso siguiente, más ambicioso, sería que **los dos jugadores evolucionen a la vez**, adaptándose el uno al otro, hasta un punto de equilibrio en el que ninguno pueda mejorar cambiando su estrategia (el **equilibrio de Nash**). Ahí el problema se vuelve más rico y el algoritmo evolutivo se luce todavía más.

Mapa mental para recordarlo



Para el detalle técnico completo, ver el README, el informe en PDF y el código comentado en `src/pushfold/` del repositorio.